Newsfeed

Saturday, 6 December 2025 10:05 AM

Social Media

Open → see posts from people you know

↓

(Newsfeed)

→ central part of social media application

Newsfeed is a constantly updating list of stories/posts on application's homepage

Requirements

Functional

→ Newsfeed must be generated from posts of people / pages that a user follows.

→ user may have many friends & might follow many pages.

→ Feed might contain image, video or text.

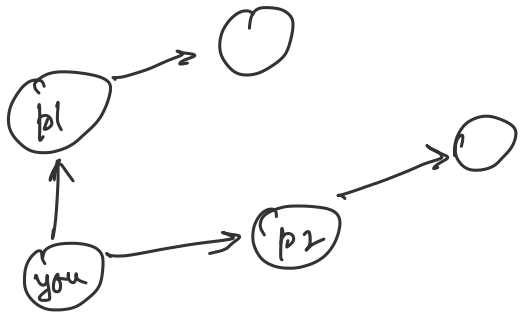
→ service should append new posts as they arrive to the newsfeed of active users.

Non-functional

→ Real time latency ~2s

newsfeed should be updated in real-time.

→ A post should not take more than 5s to make it to a user's newsfeed



Capacity Estimation

Assumptions

On average, a person has 300 friends and follows 200 pages

Traffic : Assuming 300 M daily active users and each user fetches newsfeed five times a day.

Total Traffic per day $\Rightarrow$ $\underbrace{300\ M}\ \underbrace{\times 5}$

$\Rightarrow 300 \times 10^6 \times 5$

$= 3 \times 5 \times 10^8$

$= 15 \times 10^8$

$\rightarrow \Rightarrow$ 1.5 Bn / day

Traffic per sec $\Rightarrow \dfrac{1.5 \times 10^9}{24 \times 60 \times 60} \sim 17{,}500$ req/s

Storage

Assuming we need 500 posts in a user's news feed in memory for a quick fetch.
Let's assume each post will be 1 kB in size

$\Rightarrow$ Data stored per user $\Rightarrow 500 \times 1\,kB$

$\Rightarrow 500\ kB$

Data stored for 300 M users

$\Rightarrow 500 \times 10^3 \times 300 \times 10^6$

$= 15 \times 10^4 \times 10^3 \times 10^6$

$= 15 \times 10^{13}$

$= 150 \times 10^{12}$

$\Rightarrow$ 150 TB

o $\rightarrow$ pixel (R, G, B) or (M, C, Y, B)

Hex

$1000 \times 1000 \Rightarrow 10^6$ pixels

res. $\times 3 \times 2$

System APIs

⇒ getNewsFeed ((dev_key / token),
user_id,
since_id,
count
)

return ⇒ json object with list of feed items.

Database design

→ User
→ Entity (Pages)
→ Feed Item
→ User Follow
→ Feed Media
→ Media

PK

User	
userId	: long
name	: varchar
email	: varchar
mobile	: varchar
DOB	: varchar
lastLoginAt	: timestamp

PK

Entity	
entityID	: long
name	: varchar
type	: varchar
description	: varchar
email	: varchar
mobile	: varchar
lastActivityAt	: timestamp

User Follow	
→ follower id	: long
followee id	: long
type	: 0/1

→ PK

user

user or entity

0 → user
1 → page.

Feed Item

feedItemId	: long	PK
userId	: long	
content	: varchar	
lat	: double	
long	: double	
creationAt	: timestamp	
postedBy	: long	

→ **Feed Media**

feedItemId	
mediaId	

Media

mediaId	: long
type	: Text / video / image
path	: varchar
lat	: double
long	: double
createdAt	: timestamp

High Level Design

Feed Generation

1. Retrieve all users & entities that one follows
2. Retrieve latest, posts for all these IDs
3. Rank these posts based on relevance.
4. Store in cache & return top posts for current

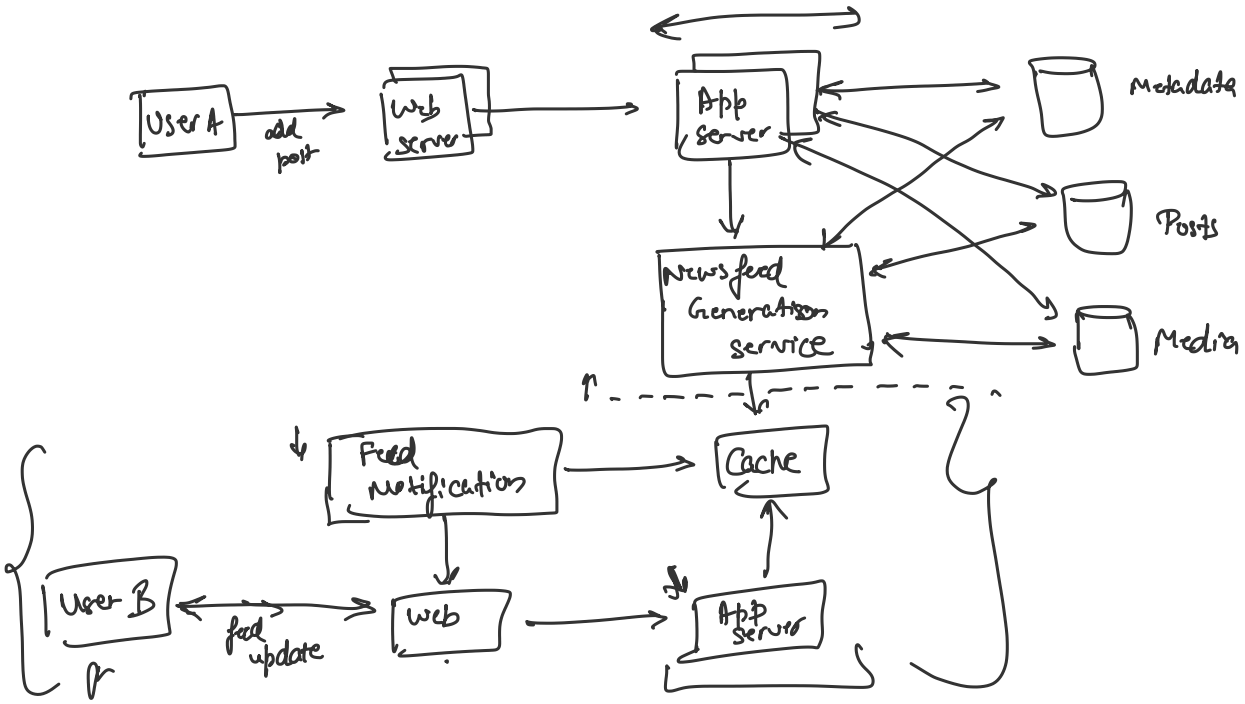
Feed Publishing.

Push

Pull → Better approach

newsfeed.

⑤ One Front-end, when user runs out of newsfeed, the next set of posts can be returned.



Component design

⇒ Feed Generation

⇒ Live generation

```
Select feed Item Id from FeedItem where user Id
IN
( select followee Id from User Follow where
  follower Id = <current-user> and type=0 )
UNION
Select feed Item Id from Feed Item where userId
IN
( Select followee Id from User Follow where
  follower Id = <current-user> and type=1 )
```

ORDER BY Creation desc,
Limit 100

Cons

→ Very slow when user has a lot of friends/ pages.

→ Leads to high latency.

→ Every status update can result in feed updates. which would lead to backlog in News feed generation.

Offline generation.

→ Use dedicated servers to generate the feed & store in cache.

→ Now, user can directly fetch from the cache instead of running live query.

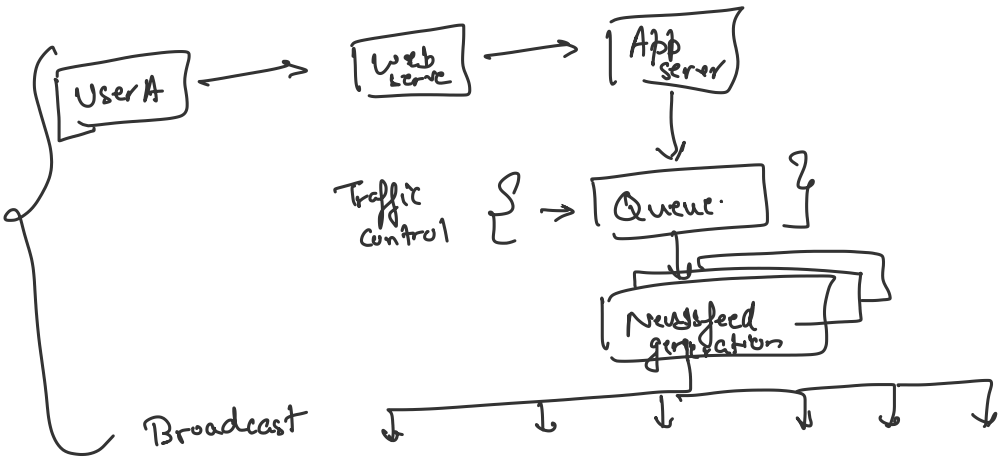
Cache Structure

```
userId: {
        LinkedHashMap < FeedItemId, FeedItem>
          items;
        Datetime  lastGenerated;
}
```

Cache
↳ 20 viewed
↳ using a paginated API we can fetch

Cache
↳ 20 viewed
news feed regenerated
cache
↳ 100 items

next 20 ... 20 viewed



(b) Feed Publishing

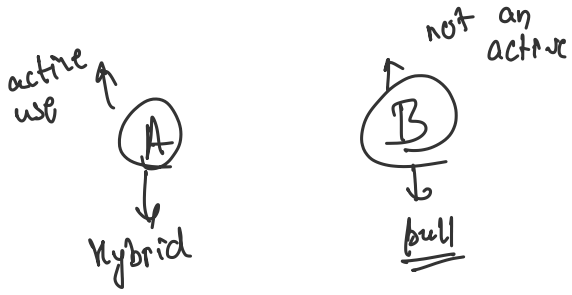
Push → Server will be pushing updates to users. (Load on servers)

Pull → client can request for new feed. there might be many request with empty responses.

Hybrid →

1. User logged in
2. Pull request for feed.
3. Create connection for push (long poll / web socket)
4. Maintain connection till user is active.
5. Repeat when user logs back in again

X → 1M followers



Feed Ranking

→ 500 posts